POLITECNICO
MILANO 1863

Ingegneria del software

Design pattern creazionali

Abstract factory

Motivazioni

- Consideriamo di voler implementare un insieme di classi dedicate alla creazione di interfacce grafiche
- Gli elementi grafici (e.g., window, button, scrollbar) possono essere creati secondo diversi stili (e.g., PM e Motif)
- Per ogni elemento grafico viene definita una classe astratta
- Per ogni stile ed elemento grafico viene definita una classa concreata

Motivazioni

- I client che creano una interfaccia grafica devono poter modificare lo stile a seconda delle esigenze, potenzialmente anche dinamicamente

- **Problema:** istanziando esplicitamente i tipi concreti di ciascuno stile (e.g., PMButton o MotifButton) si rende complessa e dispendiosa la modifica dello stile

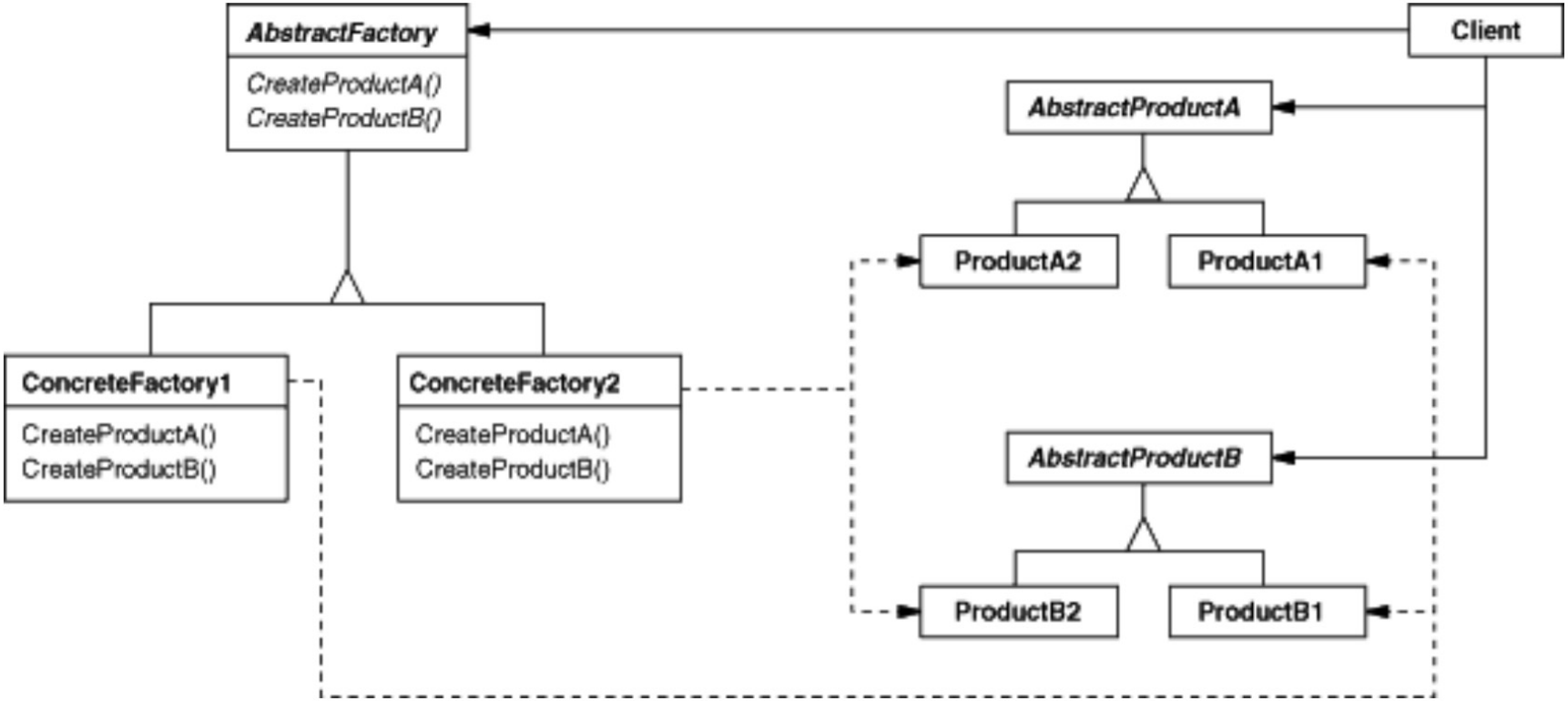
Anti-pattern

```
class ClientGUI {
        public void setupWindow(){
                Window w = new PMWindow();
                w.setSize(500, 400);
                w.addScrollbar(new PMScrollbar());
                w.addPanel(setupPanel());

        }

        public Panel setupPanel(){
                Panel p = new PMPanel();
                Button b = new PMButtion();
                p.add(b);
                …
                return p;
        }
        …
};
```

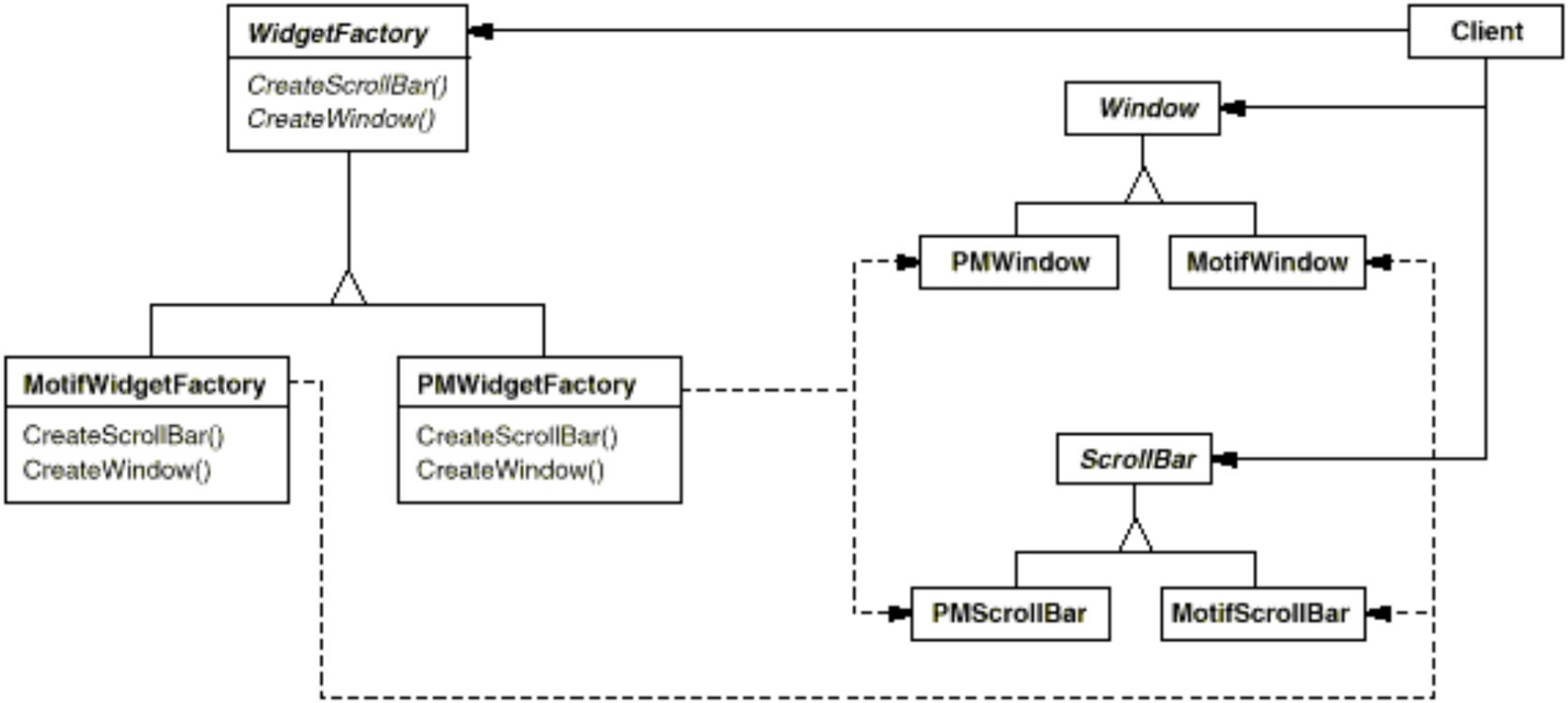
Obbiettivi

- Creare classi “factory” dedicate a generare elementi appartenti ad una gerarchia
- Una interfaccia o una classe astratta definiscono quali sono le funzionalità della factory
- I client non creano direttamente gli oggetti ma utilizzano una factory
- Modificare la tipologia di elementi da creare equivale a cambiare istanza di factory

Struttura


AbstractFactory
CreateProductA()
CreateProductB()
Client
AbstractProductA
ProductA2
ProductA1
ConcreteFactory1
CreateProductA()
CreateProductB()
ConcreteFactory2
CreateProductA()
CreateProductB()
AbstractProductB
ProductB2
ProductB1

Esempio


WidgetFactory
CreateScrollBar()
CreateWindow()
Client
Window
PMWindow
MotifWindow
MotifWidgetFactory
CreateScrollBar()
CreateWindow()
PMWidgetFactory
CreateScrollBar()
CreateWindow()
ScrollBar
PMScrollBar
MotifScrollBar

Esempio

```
abstract class WidgetFactory {
     public abstract Window createWindow();
     public abstract Panel createPanel ();
     public abstract Button createButton();
     public abstract Scrollbar createScrollbar();
     …
};
```

Esempio

```
abstract class PMWidgetFactory {
        public Window createWindow(){
                return new PMWindow();
        }
        public Panel createPanel(){
                return new PMPanel();
        }
        public Button createButton(){
                return new PMButton();
        }
        public Scrollbar createScrollbar(){
                return new PMScrollbar();
        }
        …
}
```

Esempio

```
abstract class MotifWidgetFactory {
        public Window createWindow(){
                return new MotifWindow();
        }
        public Panel createPanel(){
                return new MotifPanel();
        }
        public Button createButton(){
                return new MotifButton();
        }
        public Scrollbar createScrollbar(){
                return new MotifScrollbar();
        }
        …
}
```

Esempio

```
class ClientGUI {
        WidgetFactory factory = new PMWidgetFactory();

        public void setupWindow(){
                Window w = factory.createWindow();
                w.setSize(500, 400);
                w.addScrollbar(factory.createScrollbar());
                w.addPanel(setupPanel());

        }

        public Panel setupPanel(){
                Panel p = factory.createPanel();
                Button b = factory.createButton();
                p.add(b);
                …
                return p;
        }
        …
};
```